AI · RAG · FINTECH PROJECT

Mutual Fund RAG FAQ Assistant

A facts-only, source-backed AI assistant for HDFC mutual fund schemes — built with Retrieval-Augmented Generation, guardrails, and production deployment.

5	9	1
HDFC Schemes	Query Domains	Source Per Answer

Python | Streamlit | ChromaDB | BGE Embeddings | Groq LLM | Railway

Author: Aayush Tayal | **GitHub:** github.com/aayush13022/Mutual-Fund-RAG-FAQ-Assistant

The Problem

Investors and learners often need quick, factual answers about mutual funds — expense ratio, exit load, fund manager, benchmark, minimum SIP, and more. Generic AI chatbots can sound confident but may hallucinate, give investment advice, or answer without verifiable sources.

⚠️ Financial AI needs accuracy, scope control, and transparency — not just a large language model.

The Solution

I built a **RAG-based FAQ assistant** scoped to **5 HDFC Mutual Fund schemes** on Groww. The system retrieves relevant facts from a vector database, generates concise answers with a Groq LLM, and cites exactly one Groww source link per response.

✅ What it answers

- Expense ratio & exit load
- Fund manager & benchmark
- NAV, AUM, risk category
- Minimum SIP / lumpsum
- Tax & investment objective

🚫 What it refuses

- Investment advice
- Buy / sell / hold tips
- Return predictions
- Fund comparisons
- Out-of-scope questions

Supported Schemes

#	Scheme
1	HDFC Mid Cap Fund Direct Growth
2	HDFC Large Cap Fund Direct Growth
3	HDFC Small Cap Fund Direct Growth
4	HDFC Gold ETF Fund of Fund Direct Plan Growth
5	HDFC Defence Fund Direct Growth

Key Features

User Experience

- Streamlit chat UI with Groww branding
- "What you can ask" guide + sample questions
- 3 example question chips (auto-send)
- Persistent chat history sidebar
- New chat / Back to home actions

AI & Data Pipeline

- Daily ingestion from allowlisted Groww URLs
- Section-aware chunking & embedding
- Scheme-aware retrieval with section boost
- Intent guardrails before RAG
- Atomic blue/green index swap on re-ingestion

Architecture

```
User Browser
     │
     ▼
Streamlit App (UI + RAG in one process)
     │
     ▼
stapp/chat_handler.py  ──► Guardrails (refuse advice / comparison)
     │
     ▼
rag/generator.py
     ├── Scheme-aware retrieval (ChromaDB)
     ├── Section-boosted context selection
     └── Groq LLM answer generation
     │
     ▼
Response: Answer + Groww source link + disclaimer
```

Design Principles

Principle	Implementation
Accuracy over intelligence	Answers grounded in retrieved chunks only
Facts-only	Guardrails block advisory queries before retrieval
Source transparency	Exactly one citation link per answer
Fresh data	Daily scheduler refreshes corpus from Groww

Minimal surface	No auth, no PII, no portfolio features

Tech Stack

Layer	Technology
Language	Python 3.11+
App / UI	Streamlit
Vector Database	ChromaDB
Embeddings	BGE (small + large, chunk-aware routing)
LLM	Groq (llama-3.1-8b-instant / llama-3.3-70b)
Metadata	SQLite
Ingestion	httpx, BeautifulSoup, APScheduler
Deployment	Railway (Procfile + railway.toml)
CI / Cron	GitHub Actions (daily ingestion)
Legacy stack	FastAPI + Next.js (reference)

BUILT WITH

Python · Streamlit · ChromaDB · BGE · Groq · SQLite · APScheduler · FastAPI · Next.js · Railway · GitHub Actions

Deployment

The app ships as a **single Streamlit service** on Railway. The browser talks only to Streamlit — no CORS, no separate API hop. A persistent `/data` volume stores the ChromaDB index, metadata DB, and chat history across redeploys.

Example deployment command

```
streamlit run streamlit_app.py --server.port=$PORT --server.address=0.0.0.0
```

Example Interaction

User: What is the expense ratio of HDFC Defence Fund Direct Growth?

Assistant: The expense ratio is 0.81% (Direct Growth).

Source: groww.in/mutual-funds/hdfc-defence-fund-direct-growth · Last updated from sources · Facts-only. No investment advice.

What I Learned

Technical

- End-to-end RAG: ingest → chunk → embed → retrieve → generate
- Retrieval quality matters more than model size
- Guardrails should run *before* the LLM call
- Deployment details (memory, volumes, warmup) affect UX

Product

- Clear scope reduces user confusion and hallucinations
- Source links build trust in financial AI
- Simple UI + strict backend = better demo
- Compliance-first design is a feature, not a limitation

Project Highlights

- Full ingestion pipeline with allowlisted Groww corpus
- Scheme-aware retrieval with section-type boosting
- Chunk-aware BGE-small / BGE-large embedding routing
- Persistent chat history with sidebar navigation
- Comprehensive docs: architecture, implementation plan, deployment guide
- Test suite with pytest (handler, history, retrieval, API)

Links

Resource	URL
GitHub Repository	https://github.com/aayush13022/Mutual-Fund-RAG-FAQ-Assistant
Live Demo	Deploy on Railway / Streamlit Cloud (see repo README)

This assistant provides factual information only. It does not provide investment advice, recommendations, predictions, or financial planning guidance.

Aayush Tayal — Mutual Fund RAG FAQ Assistant
Built with: Python | Streamlit | ChromaDB | BGE | Groq | SQLite | Railway